PHARMAMOBIL

SESIÓN 08

ACTIVIDAD AUTÓNOMA · CRUD DE PRODUCTOS

Estudiante	[COMPLETAR NOMBRE COMPLETO]
Apellidos	[COMPLETAR APELLIDOS]
Fecha	[COMPLETAR FECHA]
Rama prevista	feature/crud-productos-quispe · PENDIENTE DE PUBLICAR

Estado de esta entrega

Implementación y pruebas Android host preparadas. Los resultados que requieren ejecutar la app en Android/iOS y conectar PharmaSoft quedan explícitamente pendientes; este documento no los presenta como observados.

1. Alcance y arquitectura

Se implementó el flujo de productos con separación por capas: UI y estado, casos de uso, repositorio y API HTTP Ktor. La llamada REST incluye listar con paginación, obtener, crear, actualizar y eliminar. Los DTO se mantienen separados del modelo de dominio.

Operación	Petición	HTTP real	Resultado en app
Listar	GET /api/v1/productos?pagina=0&tamanio=20	PENDIENTE	PENDIENTE
Obtener	GET /api/v1/productos/{id}	PENDIENTE	PENDIENTE
Crear	POST /api/v1/productos	PENDIENTE	PENDIENTE
Actualizar	PUT /api/v1/productos/{id}	PENDIENTE	PENDIENTE
Eliminar	DELETE /api/v1/productos/{id}	PENDIENTE	PENDIENTE

La tabla conserva los campos reales vacíos en docs/evidencias/matriz-operaciones.json. Se confirmó el contrato leyendo PharmaSoft, pero el servidor y la base Oracle no estaban disponibles para registrar respuestas HTTP reales.

2. Estados y manejo de errores

Situación	Contrato de PharmaSoft	Manejo/evidencia
HTTP 400	DTO con validationErrors por campo	Validacion; prueba MockEngine
HTTP 404	Producto no encontrado	NoEncontrado; prueba MockEngine
HTTP 409	ReglaNegocioException; nombre duplicado o producto ya inactivo	Conflicto; prueba MockEngine
DELETE	204; baja lógica estado=false, conserva fila	La app filtra inactivos tras recargar
5xx / sin conexión / timeout	Contrato estático confirmado	Mapeo implementado; falta ejecución real
Cancelación	La corrutina se cancela	CancellationException relanzada; prueba automatizada

El controlador también expone GET /api/v1/categorias. El formulario solicita un categoriaId existente. La ficha propone que un producto con ventas cause 409 al eliminar, pero el backend aplica baja lógica y no rechaza por ventas; se documenta en cambio la regla 409 real de nombre duplicado y la segunda baja del mismo ID.

3. Pruebas automatizadas disponibles

Salida guardada en docs/evidencias/salida-pruebas.txt con rutas locales redactadas. Resultado: BUILD SUCCESSFUL; Gradle reporta 56 pruebas Android host/commonTest y los XML muestran cero fallos y cero errores.

Cobertura incluye estados de pantalla, errores de validación, conservación de lista ante fallo, baja lógica con filtro de productos inactivos, cancelación y traducción de HTTP 400/404/409 con MockEngine.

La compilación :androidApp:compileDebugKotlin tuvo éxito. El APK debug no se pudo firmar porque el entorno denegó debug.keystore.lock. La prueba iOS requiere macOS y Xcode, no disponibles en este equipo. La prueba integrada Android contra PharmaSoft requiere servidor y base Oracle.

Comando ejecutado: .\gradlew.bat :shared:testAndroidHostTest

4. Bitácora de ocho escenarios

La bitácora separa expresamente respuestas esperadas por la guía y observaciones pendientes. Ejecutar cada procedimiento de docs/scripts/ y anotar fecha, plataforma, HTTP, ErrorApi, texto mostrado y ruta de la captura.

Validación de nombre y precio, producto inexistente, eliminación duplicada, regla de negocio 409, servidor detenido, timeout forzado y cancelación al salir de la pantalla.

No se inventaron mensajes, códigos, capturas ni resultados del servicio.

5. Evidencia visual por completar

Capturar cada flujo en Android y iOS, cuando el entorno esté disponible. Colocar imágenes en docs/evidencias/android/ y docs/evidencias/ios/.

CAPTURA REAL PENDIENTE

Listar productos · Android
android/productos-lista.png

CAPTURA REAL PENDIENTE

Crear / validación · Android
android/producto-crear.png

CAPTURA REAL PENDIENTE

Editar / eliminar · Android
android/producto-editar-eliminar.png

CAPTURA REAL PENDIENTE

Errores reales del servidor
errores/esc1-validacion-nombre-android.png